# ONLINE STORE (MINI E-COMMERCE)

**SQL Assignment – Complete Step-by-Step Solution**

This document contains the table creation, sample data, basic queries, joins, aggregate functions, subqueries, updates and deletes, views, stored procedures, and reports. Result tables are kept simple and black-and-white so they look similar to SQL shell output.

## 1. Create the Database

```
CREATE DATABASE online_store;
USE online_store;
```

## 2. Create Tables

### Customers

```
CREATE TABLE Customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15),
    city VARCHAR(50),
    registration_date DATE
);
```

### Categories

```
CREATE TABLE Categories (
    category_id INT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL,
    description VARCHAR(255)
);
```

### Products

```
CREATE TABLE Products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    stock_quantity INT NOT NULL,
    category_id INT,
    added_date DATE,
    FOREIGN KEY (category_id) REFERENCES Categories(category_id)
);
```

### Orders

```
CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    total_amount DECIMAL(10,2),
    status VARCHAR(30),
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);
```

### Order_Items

```
CREATE TABLE Order_Items (
    order_item_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT,
    price DECIMAL(10,2),
    subtotal DECIMAL(10,2),
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
```

## 3. Insert Sample Data

### Customers

```
INSERT INTO Customers VALUES
(1,'Rahul','rahul@gmail.com','9876543210','Hyderabad','2024-01-10'),
(2,'Priya','priya@gmail.com','9876543211','Delhi','2024-02-15'),
(3,'Arjun','arjun@gmail.com','9876543212','Hyderabad','2024-03-20'),
(4,'Sneha','sneha@gmail.com','9876543213','Mumbai','2024-04-05'),
(5,'Kiran','kiran@gmail.com','9876543214','Bangalore','2024-05-12'),
(6,'Anil','anil@gmail.com','9876543215','Chennai','2024-06-18');
```

| customer_id | name | email | phone | city | registration_date |
|---|---|---|---|---|---|
| 1 | Rahul | rahul@gmail.com | 9876543210 | Hyderabad | 2024-01-10 |
| 2 | Priya | priya@gmail.com | 9876543211 | Delhi | 2024-02-15 |
| 3 | Arjun | arjun@gmail.com | 9876543212 | Hyderabad | 2024-03-20 |
| 4 | Sneha | sneha@gmail.com | 9876543213 | Mumbai | 2024-04-05 |
| 5 | Kiran | kiran@gmail.com | 9876543214 | Bangalore | 2024-05-12 |
| 6 | Anil | anil@gmail.com | 9876543215 | Chennai | 2024-06-18 |

## Categories

```
INSERT INTO Categories VALUES
(1,'Electronics','Electronic devices and accessories'),
(2,'Clothing','Clothes and fashion products'),
(3,'Books','Books and study materials'),
(4,'Home Appliances','Appliances for home use');
```

| category_id | category_name | description |
|---|---|---|
| 1 | Electronics | Electronic devices and accessories |
| 2 | Clothing | Clothes and fashion products |
| 3 | Books | Books and study materials |
| 4 | Home Appliances | Appliances for home use |

## Products

```
INSERT INTO Products VALUES
(1,'Laptop',65000,10,1,'2024-01-15'),
(2,'Headphones',1500,30,1,'2024-02-10'),
(3,'T-Shirt',800,50,2,'2024-03-05'),
(4,'Jeans',1200,25,2,'2024-04-12'),
(5,'SQL Book',700,20,3,'2024-05-01'),
(6,'Smartphone',25000,15,1,'2024-06-10'),
(7,'Python Book',700,18,3,'2024-07-20'),
(8,'Microwave',12000,8,4,'2024-08-05');
```

| product_id | product_name | price | stock_quantity | category_id | added_date |
|---|---|---|---|---|---|
| 1 | Laptop | 65000.00 | 10 | 1 | 2024-01-15 |
| 2 | Headphones | 1500.00 | 30 | 1 | 2024-02-10 |
| 3 | T-Shirt | 800.00 | 50 | 2 | 2024-03-05 |
| 4 | Jeans | 1200.00 | 25 | 2 | 2024-04-12 |
| 5 | SQL Book | 700.00 | 20 | 3 | 2024-05-01 |
| 6 | Smartphone | 25000.00 | 15 | 1 | 2024-06-10 |
| 7 | Python Book | 700.00 | 18 | 3 | 2024-07-20 |
| 8 | Microwave | 12000.00 | 8 | 4 | 2024-08-05 |

## Orders

```
INSERT INTO Orders VALUES
(101,1,'2024-07-01',66500,'Completed'),
(102,2,'2024-07-05',26800,'Completed'),
(103,3,'2024-07-10',2000,'Completed'),
(104,4,'2024-07-15',12000,'Completed'),
(105,1,'2024-07-20',3000,'Cancelled'),
(106,5,'2024-08-02',26200,'Completed');
```

| order_id | customer_id | order_date | total_amount | status |
|---|---|---|---|---|
| 101 | 1 | 2024-07-01 | 66500.00 | Completed |
| 102 | 2 | 2024-07-05 | 26800.00 | Completed |
| 103 | 3 | 2024-07-10 | 2000.00 | Completed |

| order_id | customer_id | order_date | total_amount | status |
|---|---|---|---|---|
| 104 | 4 | 2024-07-15 | 12000.00 | Completed |
| 105 | 1 | 2024-07-20 | 3000.00 | Cancelled |
| 106 | 5 | 2024-08-02 | 26200.00 | Completed |

### Order_Items

```
INSERT INTO Order_Items VALUES
(1,101,1,1,65000,65000),
(2,101,2,1,1500,1500),
(3,102,6,1,25000,25000),
(4,102,2,1,1500,1500),
(5,102,5,1,700,700),
(6,103,3,1,800,800),
(7,103,4,1,1200,1200),
(8,104,8,1,12000,12000),
(9,105,2,2,1500,3000),
(10,106,6,1,25000,25000),
(11,106,5,1,700,700),
(12,106,7,1,1200,1200);
```

| order_item_id | order_id | product_id | quantity | price | subtotal |
|---|---|---|---|---|---|
| 1 | 101 | 1 | 1 | 65000.00 | 65000.00 |
| 2 | 101 | 2 | 1 | 1500.00 | 1500.00 |
| 3 | 102 | 6 | 1 | 25000.00 | 25000.00 |
| 4 | 102 | 2 | 1 | 1500.00 | 1500.00 |
| 5 | 102 | 5 | 1 | 700.00 | 700.00 |
| 6 | 103 | 3 | 1 | 800.00 | 800.00 |
| 7 | 103 | 4 | 1 | 1200.00 | 1200.00 |
| 8 | 104 | 8 | 1 | 12000.00 | 12000.00 |
| 9 | 105 | 2 | 2 | 1500.00 | 3000.00 |
| 10 | 106 | 6 | 1 | 25000.00 | 25000.00 |
| 11 | 106 | 5 | 1 | 700.00 | 700.00 |
| 12 | 106 | 7 | 1 | 1200.00 | 1200.00 |

# 4. Basic Queries

## 4.1 Products with price greater than 1000

```
SELECT * FROM Products
WHERE price > 1000;
```

| product_id | product_name | price | stock_quantity | category_id | added_date |
|---|---|---|---|---|---|
| 1 | Laptop | 65000.00 | 10 | 1 | 2024-01-15 |
| 2 | Headphones | 1500.00 | 30 | 1 | 2024-02-10 |
| 4 | Jeans | 1200.00 | 25 | 2 | 2024-04-12 |
| 6 | Smartphone | 25000.00 | 15 | 1 | 2024-06-10 |
| 8 | Microwave | 12000.00 | 8 | 4 | 2024-08-05 |

## 4.2 Customers from Hyderabad

```
SELECT * FROM Customers
WHERE city = 'Hyderabad';
```

| customer_id | name | email | phone | city | registration_date |
|---|---|---|---|---|---|
| 1 | Rahul | rahul@gmail.com | 9876543210 | Hyderabad | 2024-01-10 |
| 3 | Arjun | arjun@gmail.com | 9876543212 | Hyderabad | 2024-03-20 |

## 4.3 Products from Electronics category

```
SELECT p.*
FROM Products p
JOIN Categories c ON p.category_id = c.category_id
```

```
WHERE c.category_name = 'Electronics';
```

| product_id | product_name | price | stock_quantity | category_id | added_date |
|---|---|---|---|---|---|
| 1 | Laptop | 65000.00 | 10 | 1 | 2024-01-15 |
| 2 | Headphones | 1500.00 | 30 | 1 | 2024-02-10 |
| 6 | Smartphone | 25000.00 | 15 | 1 | 2024-06-10 |

### 4.4 Products added after 2024-01-01

```
SELECT * FROM Products
WHERE added_date > '2024-01-01';
```

All eight sample products were added after 2024-01-01, so all eight rows are returned.

## 5. Joins

### 5.1 Customer names with their orders

```
SELECT c.name, o.order_id, o.order_date, o.total_amount, o.status
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id;
```

| name | order_id | order_date | total_amount | status |
|---|---|---|---|---|
| Rahul | 101 | 2024-07-01 | 66500.00 | Completed |
| Priya | 102 | 2024-07-05 | 26800.00 | Completed |
| Arjun | 103 | 2024-07-10 | 2000.00 | Completed |
| Sneha | 104 | 2024-07-15 | 12000.00 | Completed |
| Rahul | 105 | 2024-07-20 | 3000.00 | Cancelled |
| Kiran | 106 | 2024-08-02 | 26200.00 | Completed |

### 5.2 Products with quantity ordered

```
SELECT p.product_name, oi.quantity
FROM Products p
JOIN Order_Items oi ON p.product_id = oi.product_id;
```

| product_name | quantity |
|---|---|
| Laptop | 1 |
| Headphones | 1 |
| Smartphone | 1 |
| Headphones | 1 |
| SQL Book | 1 |
| T-Shirt | 1 |
| Jeans | 1 |
| Microwave | 1 |
| Headphones | 2 |
| Smartphone | 1 |
| SQL Book | 1 |
| Python Book | 1 |

### 5.3 Order details with customer information

```
SELECT o.order_id, c.name, c.email, o.order_date, o.total_amount, o.status
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id;
```

| order_id | name | email | order_date | total_amount | status |
|---|---|---|---|---|---|
| 101 | Rahul | rahul@gmail.com | 2024-07-01 | 66500.00 | Completed |
| 102 | Priya | priya@gmail.com | 2024-07-05 | 26800.00 | Completed |
| 103 | Arjun | arjun@gmail.com | 2024-07-10 | 2000.00 | Completed |
| 104 | Sneha | sneha@gmail.com | 2024-07-15 | 12000.00 | Completed |
| 105 | Rahul | rahul@gmail.com | 2024-07-20 | 3000.00 | Cancelled |

| order_id | name | email | order_date | total_amount | status |
|---|---|---|---|---|---|
| 106 | Kiran | kiran@gmail.com | 2024-08-02 | 26200.00 | Completed |

### 5.4 Products with their categories

```
SELECT p.product_name, c.category_name
FROM Products p
JOIN Categories c ON p.category_id = c.category_id;
```

| product_name | category_name |
|---|---|
| Laptop | Electronics |
| Headphones | Electronics |
| T-Shirt | Clothing |
| Jeans | Clothing |
| SQL Book | Books |
| Smartphone | Electronics |
| Python Book | Books |
| Microwave | Home Appliances |

### 5.5 Customers and products purchased

```
SELECT DISTINCT c.name, p.product_name
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
JOIN Order_Items oi ON o.order_id = oi.order_id
JOIN Products p ON oi.product_id = p.product_id
WHERE o.status = 'Completed';
```

| name | product_name |
|---|---|
| Rahul | Laptop |
| Rahul | Headphones |
| Priya | Smartphone |
| Priya | Headphones |
| Priya | SQL Book |
| Arjun | T-Shirt |
| Arjun | Jeans |
| Sneha | Microwave |
| Kiran | Smartphone |
| Kiran | SQL Book |
| Kiran | Python Book |

## 6. Aggregate Functions

### 6.1 Count total products

```
SELECT COUNT(*) AS total_products FROM Products;
```

| total_products |
|---|
| 8 |

### 6.2 Average product price

```
SELECT AVG(price) AS average_price FROM Products;
```

| average_price |
|---|
| 13718.7500 |

### 6.3 Total completed sales

```
SELECT SUM(total_amount) AS total_sales
FROM Orders
WHERE status = 'Completed';
```

| total_sales |
|---|
| 133500.00 |

### 6.4 Count completed orders

```
SELECT COUNT(*) AS completed_orders
FROM Orders
WHERE status = 'Completed';
```

| completed_orders |
|---|
| 5 |

### 6.5 Highest product price

```
SELECT MAX(price) AS highest_price FROM Products;
```

| highest_price |
|---|
| 65000.00 |

## 7. Subqueries

### 7.1 Customers who have no orders

```
SELECT *
FROM Customers
WHERE customer_id NOT IN (
    SELECT customer_id FROM Orders
);
```

| customer_id | name | email | phone | city | registration_date |
|---|---|---|---|---|---|
| 6 | Anil | anil@gmail.com | 9876543215 | Chennai | 2024-06-18 |

### 7.2 Products that were never ordered

```
SELECT *
FROM Products
WHERE product_id NOT IN (
    SELECT product_id FROM Order_Items
);
```

Result: no rows, because every sample product appears in Order_Items.

### 7.3 Products priced above the average

```
SELECT product_name, price
FROM Products
WHERE price > (
    SELECT AVG(price) FROM Products
);
```

| product_name | price |
|---|---|
| Laptop | 65000.00 |
| Smartphone | 25000.00 |

### 7.4 Customers whose completed spending is above the average customer spending

```
SELECT c.name, SUM(o.total_amount) AS total_spending
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
WHERE o.status = 'Completed'
GROUP BY c.customer_id, c.name
HAVING SUM(o.total_amount) > (
    SELECT AVG(customer_total)
    FROM (
        SELECT SUM(total_amount) AS customer_total
        FROM Orders
        WHERE status = 'Completed'
        GROUP BY customer_id
    ) x
);
```

| name | total_spending |
|---|---|
| Rahul | 78500.00 |

The average completed spending among customers who placed completed orders is 33125.00.

### 7.5 Completed orders above the average completed order amount

```
SELECT order_id, customer_id, total_amount
FROM Orders
WHERE status = 'Completed'
AND total_amount > (
    SELECT AVG(total_amount)
    FROM Orders
    WHERE status = 'Completed'
);
```

| order_id | customer_id | total_amount |
|---|---|---|
| 101 | 1 | 66500.00 |
| 102 | 2 | 26800.00 |

The average completed order amount is 26700.00.

## 8. Updates and Deletes

### 8.1 Update a product price

```
UPDATE Products
SET price = 1800
WHERE product_id = 2;

SELECT product_id, product_name, price
FROM Products
WHERE product_id = 2;
```

| product_id | product_name | price |
|---|---|---|
| 2 | Headphones | 1800.00 |

### 8.2 Update a customer email

```
UPDATE Customers
SET email = 'rahul.new@gmail.com'
WHERE customer_id = 1;

SELECT customer_id, name, email
FROM Customers
WHERE customer_id = 1;
```

| customer_id | name | email |
|---|---|---|
| 1 | Rahul | rahul.new@gmail.com |

### 8.3 Update a product category

```
UPDATE Products
SET category_id = 4
WHERE product_id = 7;

SELECT product_id, product_name, category_id
FROM Products
WHERE product_id = 7;
```

| product_id | product_name | category_id |
|---|---|---|
| 7 | Python Book | 4 |

### 8.4 Delete cancelled orders

```
DELETE FROM Order_Items
WHERE order_id IN (
    SELECT order_id FROM Orders
    WHERE status = 'Cancelled'
);
```

```
DELETE FROM Orders
WHERE status = 'Cancelled';
```

The cancelled order 105 and its order item are removed. Child rows are deleted first because of the foreign key.

### 8.5 Delete customers who have no orders

```
DELETE FROM Customers
WHERE customer_id NOT IN (
    SELECT customer_id FROM Orders
);
```

After deleting cancelled order 105, Anil is the customer with no remaining order and can be deleted.

## 9. Views

### 9.1 Customer orders view

```
CREATE VIEW customer_orders AS
SELECT c.customer_id, c.name, o.order_id,
       o.order_date, o.total_amount, o.status
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id;

SELECT * FROM customer_orders;
```

This view combines customer and order information in one reusable result.

### 9.2 Product and category view

```
CREATE VIEW product_category AS
SELECT p.product_id, p.product_name,
       p.price, c.category_name
FROM Products p
JOIN Categories c ON p.category_id = c.category_id;

SELECT * FROM product_category;
```

| product_id | product_name | price | category_name |
|---|---|---|---|
| 1 | Laptop | 65000.00 | Electronics |
| 2 | Headphones | 1800.00 | Electronics |
| 3 | T-Shirt | 800.00 | Clothing |
| 4 | Jeans | 1200.00 | Clothing |
| 5 | SQL Book | 700.00 | Books |
| 6 | Smartphone | 25000.00 | Electronics |
| 7 | Python Book | 700.00 | Home Appliances |
| 8 | Microwave | 12000.00 | Home Appliances |

### 9.3 Order item details view

```
CREATE VIEW order_item_details AS
SELECT oi.order_item_id, oi.order_id,
       p.product_name, oi.quantity,
       oi.price, oi.subtotal
FROM Order_Items oi
JOIN Products p ON oi.product_id = p.product_id;

SELECT * FROM order_item_details;
```

### 9.4 Customer total spending view

```
CREATE VIEW customer_total_spending AS
SELECT c.customer_id, c.name,
       COALESCE(SUM(
           CASE WHEN o.status = 'Completed'
                THEN o.total_amount ELSE 0 END
       ),0) AS total_spending
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;

SELECT * FROM customer_total_spending;
```

| customer_id | name | total_spending |
|---|---|---|
| 1 | Rahul | 78500.00 |
| 2 | Priya | 26800.00 |
| 3 | Arjun | 2000.00 |
| 4 | Sneha | 12000.00 |
| 5 | Kiran | 26200.00 |

### 9.5 Best-selling products view

```
CREATE VIEW best_selling_products AS
SELECT p.product_id, p.product_name,
       SUM(oi.quantity) AS total_quantity_sold
FROM Products p
JOIN Order_Items oi ON p.product_id = oi.product_id
JOIN Orders o ON oi.order_id = o.order_id
WHERE o.status = 'Completed'
GROUP BY p.product_id, p.product_name;

SELECT * FROM best_selling_products
ORDER BY total_quantity_sold DESC, product_name ASC;
```

| product_id | product_name | total_quantity_sold |
|---|---|---|
| 2 | Headphones | 2 |
| 5 | SQL Book | 2 |
| 6 | Smartphone | 2 |
| 4 | Jeans | 1 |
| 1 | Laptop | 1 |
| 3 | T-Shirt | 1 |
| 7 | Python Book | 1 |
| 8 | Microwave | 1 |

## 10. Stored Procedures

### 10.1 Insert a new customer

```
DELIMITER //

CREATE PROCEDURE insert_customer(
    IN p_name VARCHAR(100),
    IN p_email VARCHAR(100),
    IN p_phone VARCHAR(15),
    IN p_city VARCHAR(50),
    IN p_registration_date DATE
)
BEGIN
    INSERT INTO Customers
    VALUES (
        (SELECT COALESCE(MAX(customer_id),0) + 1 FROM Customers),
        p_name, p_email, p_phone, p_city, p_registration_date
    );
END //

DELIMITER ;

CALL insert_customer(
    'Vijay',
    'vijay@gmail.com',
    '9876543220',
    'Pune',
    '2024-09-01'
);

SELECT * FROM Customers
WHERE name = 'Vijay';
```

| customer_id | name | email | phone | city | registration_date |
|---|---|---|---|---|---|
| 6 | Vijay | vijay@gmail.com | 9876543220 | Pune | 2024-09-01 |

## 10.2 Update product price

```
DELIMITER //

CREATE PROCEDURE update_product_price(
    IN p_product_id INT,
    IN p_new_price DECIMAL(10,2)
)
BEGIN
    UPDATE Products
    SET price = p_new_price
    WHERE product_id = p_product_id;
END //

DELIMITER ;

CALL update_product_price(2, 2000);

SELECT product_id, product_name, price
FROM Products
WHERE product_id = 2;
```

| product_id | product_name | price |
|---|---|---|
| 2 | Headphones | 2000.00 |

## 10.3 Display customer orders

```
DELIMITER //

CREATE PROCEDURE display_customer_orders(
    IN p_customer_id INT
)
BEGIN
    SELECT order_id, order_date, total_amount, status
    FROM Orders
    WHERE customer_id = p_customer_id;
END //

DELIMITER ;

CALL display_customer_orders(1);
```

| order_id | order_date | total_amount | status |
|---|---|---|---|
| 101 | 2024-07-01 | 66500.00 | Completed |
| 105 | 2024-07-20 | 3000.00 | Cancelled |

## 10.4 Calculate total sales

```
DELIMITER //

CREATE PROCEDURE total_sales()
BEGIN
    SELECT SUM(total_amount) AS total_sales
    FROM Orders
    WHERE status = 'Completed';
END //

DELIMITER ;

CALL total_sales();
```

| total_sales |
|---|
| 133500.00 |

## 10.5 List products by category

```
DELIMITER //

CREATE PROCEDURE products_by_category(
    IN p_category_id INT
)
BEGIN
    SELECT product_id, product_name, price, stock_quantity
    FROM Products
    WHERE category_id = p_category_id;
```

```
END //

DELIMITER ;

CALL products_by_category(1);
```

| product_id | product_name | price | stock_quantity |
|---|---|---|---|
| 1 | Laptop | 65000.00 | 10 |
| 2 | Headphones | 2000.00 | 30 |
| 6 | Smartphone | 25000.00 | 15 |

# 11. Reports and Analysis

## 11.1 Top 5 best-selling products

```
SELECT p.product_name,
       SUM(oi.quantity) AS total_quantity_sold
FROM Products p
JOIN Order_Items oi ON p.product_id = oi.product_id
JOIN Orders o ON oi.order_id = o.order_id
WHERE o.status = 'Completed'
GROUP BY p.product_id, p.product_name
ORDER BY total_quantity_sold DESC, p.product_name ASC
LIMIT 5;
```

| product_name | total_quantity_sold |
|---|---|
| Headphones | 2 |
| SQL Book | 2 |
| Smartphone | 2 |
| Jeans | 1 |
| Laptop | 1 |

## 11.2 Most frequent customers

```
SELECT c.name, COUNT(o.order_id) AS order_count
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
WHERE o.status = 'Completed'
GROUP BY c.customer_id, c.name
ORDER BY order_count DESC, c.name ASC;
```

| name | order_count |
|---|---|
| Rahul | 1 |
| Arjun | 1 |
| Kiran | 1 |
| Priya | 1 |
| Sneha | 1 |

All customers with completed orders have one completed order in the sample data.

## 11.3 Monthly sales

```
SELECT DATE_FORMAT(order_date,'%Y-%m') AS month,
       SUM(total_amount) AS monthly_sales
FROM Orders
WHERE status = 'Completed'
GROUP BY DATE_FORMAT(order_date,'%Y-%m')
ORDER BY month;
```

| month | monthly_sales |
|---|---|
| 2024-07 | 95300.00 |
| 2024-08 | 26200.00 |

## 11.4 Highest-revenue category

```
SELECT c.category_name,
       SUM(oi.subtotal) AS category_revenue
```

```
FROM Categories c
JOIN Products p ON c.category_id = p.category_id
JOIN Order_Items oi ON p.product_id = oi.product_id
JOIN Orders o ON oi.order_id = o.order_id
WHERE o.status = 'Completed'
GROUP BY c.category_id, c.category_name
ORDER BY category_revenue DESC
LIMIT 1;
```

| category_name | category_revenue |
|---|---|
| Electronics | 118000.00 |

### 11.5 Customer spending analysis

```
SELECT c.name,
       COALESCE(SUM(
           CASE WHEN o.status = 'Completed'
                THEN o.total_amount ELSE 0 END
       ),0) AS total_spending
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_spending DESC;
```

| name | total_spending |
|---|---|
| Rahul | 78500.00 |
| Priya | 26800.00 |
| Kiran | 26200.00 |
| Sneha | 12000.00 |
| Arjun | 2000.00 |
| Anil | 0.00 |

## 12. Important Data Check

There is a small inconsistency in the sample data. Order 102 has total_amount = 26800, while its Order_Items add up to 27200. Order 106 has total_amount = 26200, while its Order_Items add up to 26900. Therefore, reports based on Orders.total_amount and reports based on Order_Items.subtotal can produce different totals.

Using Orders.total_amount, completed sales are 133500.00. Using completed Order_Items.subtotal, the item-level revenue is 134600.00. This is useful to identify because a real e-commerce system should keep order totals consistent with its order-item details.

### Correct item-level revenue check

```
SELECT SUM(oi.subtotal) AS item_level_sales
FROM Order_Items oi
JOIN Orders o ON oi.order_id = o.order_id
WHERE o.status = 'Completed';
```

| item_level_sales |
|---|
| 134600.00 |

## 13. Complete SQL Script

The following section is a compact version of the main SQL setup and queries. It can be copied into MySQL Workbench after reviewing the execution order above.

```
CREATE DATABASE online_store;
USE online_store;

CREATE TABLE Customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15),
    city VARCHAR(50),
    registration_date DATE
);
```

```
CREATE TABLE Categories (
    category_id INT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL,
    description VARCHAR(255)
);

CREATE TABLE Products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    stock_quantity INT NOT NULL,
    category_id INT,
    added_date DATE,
    FOREIGN KEY (category_id) REFERENCES Categories(category_id)
);

CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    total_amount DECIMAL(10,2),
    status VARCHAR(30),
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

CREATE TABLE Order_Items (
    order_item_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT,
    price DECIMAL(10,2),
    subtotal DECIMAL(10,2),
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

INSERT INTO Customers VALUES
(1,'Rahul','rahul@gmail.com','9876543210','Hyderabad','2024-01-10'),
(2,'Priya','priya@gmail.com','9876543211','Delhi','2024-02-15'),
(3,'Arjun','arjun@gmail.com','9876543212','Hyderabad','2024-03-20'),
(4,'Sneha','sneha@gmail.com','9876543213','Mumbai','2024-04-05'),
(5,'Kiran','kiran@gmail.com','9876543214','Bangalore','2024-05-12'),
(6,'Anil','anil@gmail.com','9876543215','Chennai','2024-06-18');

INSERT INTO Categories VALUES
(1,'Electronics','Electronic devices and accessories'),
(2,'Clothing','Clothes and fashion products'),
(3,'Books','Books and study materials'),
(4,'Home Appliances','Appliances for home use');

INSERT INTO Products VALUES
(1,'Laptop',65000,10,1,'2024-01-15'),
(2,'Headphones',1500,30,1,'2024-02-10'),
(3,'T-Shirt',800,50,2,'2024-03-05'),
(4,'Jeans',1200,25,2,'2024-04-12'),
(5,'SQL Book',700,20,3,'2024-05-01'),
(6,'Smartphone',25000,15,1,'2024-06-10'),
(7,'Python Book',700,18,3,'2024-07-20'),
(8,'Microwave',12000,8,4,'2024-08-05');

INSERT INTO Orders VALUES
(101,1,'2024-07-01',66500,'Completed'),
(102,2,'2024-07-05',26800,'Completed'),
(103,3,'2024-07-10',2000,'Completed'),
(104,4,'2024-07-15',12000,'Completed'),
(105,1,'2024-07-20',3000,'Cancelled'),
(106,5,'2024-08-02',26200,'Completed');

INSERT INTO Order_Items VALUES
(1,101,1,1,65000,65000),
(2,101,2,1,1500,1500),
(3,102,6,1,25000,25000),
(4,102,2,1,1500,1500),
(5,102,5,1,700,700),
(6,103,3,1,800,800),
(7,103,4,1,1200,1200),
(8,104,8,1,12000,12000),
(9,105,2,2,1500,3000),
(10,106,6,1,25000,25000),
(11,106,5,1,700,700),
```

```
(12,106,7,1,1200,1200);

SELECT * FROM Products WHERE price > 1000;
SELECT * FROM Customers WHERE city = 'Hyderabad';

SELECT p.*
FROM Products p
JOIN Categories c ON p.category_id = c.category_id
WHERE c.category_name = 'Electronics';

SELECT * FROM Products WHERE added_date > '2024-01-01';

SELECT c.name, o.order_id, o.order_date, o.total_amount, o.status
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id;

SELECT p.product_name, oi.quantity
FROM Products p
JOIN Order_Items oi ON p.product_id = oi.product_id;

SELECT o.order_id, c.name, c.email, o.order_date, o.total_amount, o.status
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id;

SELECT p.product_name, c.category_name
FROM Products p
JOIN Categories c ON p.category_id = c.category_id;

SELECT COUNT(*) AS total_products FROM Products;
SELECT AVG(price) AS average_price FROM Products;

SELECT SUM(total_amount) AS total_sales
FROM Orders
WHERE status = 'Completed';

SELECT MAX(price) AS highest_price FROM Products;

SELECT *
FROM Customers
WHERE customer_id NOT IN (SELECT customer_id FROM Orders);

SELECT *
FROM Products
WHERE product_id NOT IN (SELECT product_id FROM Order_Items);

SELECT product_name, price
FROM Products
WHERE price > (SELECT AVG(price) FROM Products);

SELECT order_id, customer_id, total_amount
FROM Orders
WHERE status = 'Completed'
AND total_amount > (
    SELECT AVG(total_amount)
    FROM Orders
    WHERE status = 'Completed'
);
```

## 14. Conclusion

This mini e-commerce database demonstrates the main SQL operations used in a practical online store: table creation, primary and foreign keys, inserting data, filtering, joins, aggregate functions, subqueries, updates, deletes, views, stored procedures, and business reports.

The sample data also demonstrates why checking consistency between Orders.total_amount and Order_Items.subtotal is important in a real database.